

Exercise Sheet 2

Writing good Bash scripts

23 June 2026

Exercise 1

{Study}

Refresh your mind

As good start of the exercise session, you are encouraged to go through the material presented in the morning once more alone. Focus on the examples and try to understand all the details. Do not hesitate to try to copy the code there to your terminal and play around with it.

Exercise 2

{Instructive}

Loops, loops everywhere

Consider the following tasks and write for each a short script. Use the `mv` command to rename files.

1. Given two extensions, rename all files in the present folder having the first extension, changing it in the second provided one.
2. Suppose to have files in a folder with spaces in the name. Create a script to rename them, replacing each space with an underscore. Use the `touch` command with the brace-expansion feature in order to create a test environment to test your script.
3. Create an empty folder and move into it. Create some empty files via

```
touch conf.{1000..200000..5000},save prng.{1000..180000..3000},save
```

Implement a script that

- (a) deletes `conf` or `prng` files, if a `prng` or `conf` with the same number is not present;
- (b) find the largest number for which both a `conf` and a `prng` file exist.

Exercise 3

{Instructive}

The power of arrays

Arrays play a very important role in Bash scripting and they are too often forgotten. To warm up and also explore the array notation think about

- how to find the longest entry of an array?
- how to find the maximum of an array with *numeric* entries?
- how to find only common entries in two arrays?
- how to check if an array is sparse?
- how to sort an array?^a

Test your code with the following variables:

```
stringArray=( $' Hello\nworld\n' 'Hello' 'Bye' 'Goodbye everyone' 'Wow' )
numberArray=( 200 101 31 5 7 11 )
sparseArray=( [10]=$' Hello\nworld\n' [3]='Bye' [5]='Ciao' )
```

Once you feel comfortable with the array syntax, tackle the following problems.

1. Write a Bash script to make a report of the files in the present folder, counting them by extension. In order to test your script, create a test folder where you can create some files via

```
for ext in jpg png eps pdf txt odt tex; do
  for ((index=1; index<=$(shuf -i 3-9 -n 1); index++)); do
    touch file_${index}.${ext}
  done
done; unset -v 'ext' 'index'
```

2. Write a script that given integers on the command line parameters, calculates their **greatest common divisors** and their **least common multiple**, using the prime factorisations. Take a look to the `factor` command.

^aTo properly tackle this task using the `sort` command, you need Day 3 lecture. Come back here afterwards, then.

Exercise 4

{ Inspirational }

The most flexible command line parser

Most Bash commands can be comfortably used thanks to command line options, which usually have a short and a long version. The `-h` or `--help` option is the starting point when you want to discover what a script does. Therefore, **any script** you will write **should have a `--help` option** to provide information about itself. In this exercise you will write by hand your command line parser, which you will use over and over again in all your scripts!

In general a command line parser is a block of code, which processes the command line parameters, doing some actions (typically setting global variables for the rest of the script). Here reasonable requirements you should fulfil.

1. Validate as often as possible the value of each option, checking for missing values as well.
2. By convention, `--` should separate script options from file(s) or, more in general, a list of “known objects”.
3. If an option is not recognised, an error should be given.
4. Although it is nothing more than a common habit, short options have only one dash followed by one character only, while long options start with a double dash.
5. If the user asked for the help, all other (potentially wrong) options should be ignored.

In this exercise, just as practice example, you should implement the following options.

`-f, --file`

An existing file to be stored in a variable.

`-b, --beta`

A positive number with at maximum 4 decimal digits. The number without + and with exactly 4 decimal digits should be stored in a variable.

`-v, --verbose`

An option without value which should be used to activate a verbose mode. If given it should set a global variable to `'TRUE'`, which otherwise should be set to `'FALSE'`.

`-s, --sizes`

An option that can take an arbitrary number of positive integer numbers, which should be stored in an array.

`-a, --aspectRatios`

Exactly the same as the `-s` option but values should be stored in a different array.

`-r, --range`

An option to specify a lower and an upper bound to be stored in two variables.

`-w, --walltime`

A string in the format `d-hh:mm:ss` which should be stored in a variable. The user can either specify it like that or specify integers followed by `d, h, m, s` (e.g. `1d4h`).

`--`

If present, a double dash indicates that what follows should be a list of files. Store them in an array, without checking if they exist.

Requirement: The options `-s` and the option `-a` are mutually exclusive, i.e. cannot be specified at the same time. Approach the problem supposing to have a larger set of mutually exclusive options.

Bonus track: How would you allow the user to combine multiple short options? In this case the user might specify e.g. `-vs 7 13` instead of `-v -s 7 13`.

Exercise 5

{Instructive}

A completely meaningless game (for humans)

For sure you will know the game in which you have to find the word not belonging to a given group of words. In this exercise we want to implement a variation of it... let's say for robots who cannot find a logic and just guess. The final game should ask the user interactively how many words should be in the group, display a group of random words and then ask which one does not belong to the group and repeat the question until the user guesses correctly. A couple of remarks:

- Draw your words randomly from the system dictionary which should be somewhere like `/usr/share/dict/words`.
- Use an appropriate Bash keyword to let the user choose the word in the group.

Exercise 6

{Inspirational}

Shell options and GNU coreutils: Inquire about files

Write a Bash script that

1. takes one or more file extensions on the command line;
2. finds all the files from the invoking position with such an extension (also in sub-folders);
3. prints a small report about each extension, specifying
 - the number of existing files;
 - the size of the largest file;
 - The total number of lines of all files.

You learnt today about *shell options* and *GNU core utilities*. Use this exercise to do some practice with them. In particular, write this script enabling `nounset` and `errexit` options via

```
set -o nounset -o errexit # equivalent to 'set -u -e' or 'set -ue'
```

at the beginning of your script after the shebang.

Attention: You might be surprised from some strange shell behaviour and this might be connected to the enabled shell options. Feel free to ask if something strange occurs. We will discuss this aspect in detail on Day 4.

Make sure to test your script, ideally in a temporary folder created for this purpose. Did you consider the possibility no files of the given extension exist?

Exercise 7

{Instructive}

Counting words

Write a script that repeatedly reads a word from standard input and keeps track of how many times each word has been entered.

The following special commands should be recognised using a `case` statement:

```
show or s → Print all words together with their counts.
reset or r → Reset all counters.
quit or q → Exit the script.
```

Any other input should be treated as a word whose counter must be increased.

Optional: If the user gives multiple words, split them on spaces and process them all.

Hint: Consider using a construct of the following form:

```
while read -r -p " > " word; do
# ...
done
```